AppWeaver: Agentic Discovery and Composition of O-RAN Apps for AI-Native 6G

Yongchao Dang, Tao Chen, and Riku Jäntti

Abstract—Open RAN (O-RAN) opens the radio access network to a growing ecosystem of third-party rApps and xApps, allowing radio resource optimization through standardized Non-Real-Time (Non-RT) and Near-Real-Time (Near-RT) RAN Intelligent Controllers (RICs). However, building O-RAN services for emerging 6G verticals, such as connected vehicles and immersive VR/XR, remains difficult because operators must still manually discover suitable applications, understand their dependencies, configure and validate them, and coordinate them into end-to-end workflows. This article introduces AppWeaver, an agentic-AI framework that orchestrates end-to-end O-RAN workflows by using LLM-based agents to interpret operator intent, query an Agent Card registry that describes each xApp and rApp in terms of its functions, properties, and interfaces, reason about their capabilities and dependencies, and compose them into ordered, service-specific workflows. In addition, AppWeaver confines LLM agents to workflow-level orchestration over existing xApps and rApps, rather than direct RAN control, preserving a clear control-plane boundary in which native RIC applications execute low-level RAN control and monitoring actions, while agent-generated workflows are evaluated against operator policies, security constraints, validation outcomes, and rollback requirements. The resulting workflows are orchestrated with LangGraph, coordinated via Agent-to-Agent (A2A) communication, connected to xApps and rApps through Model Context Protocol (MCP) tool bindings, and validated in a FlexRIC and OpenAirInterface test environment.

Index Terms—O-RAN, agentic AI, large language models (LLMs), workflow composition.

I. Introduction

Upcoming 6G networks will evolve into programmable, artificial intelligence (AI)-native platforms for immersive, sensing, and edge-intelligence services, whose diverse requirements demand flexible and service-aware RAN automation [1]. To provide this flexibility, the O-RAN ALLIANCE[1] defines a disaggregated and programmable RAN architecture with standardized open interfaces and hierarchical RAN Intelligent Controllers (RICs), where rApps in the Non-Real-Time (Non-RT) RIC support long-term optimization and policy guidance, while xApps in the Near-Real-Time (Near-RT) RIC provide near-real-time RAN control [2]. This openness reduces vendor lock-in and enables an open ecosystem of interoperable third-party xApps and rApps for service-specific RAN automation, providing end users with more adaptive, service-aware connectivity.

Building on this openness, Hexa-X-II[2] advances intent-based management for 6G operations, where operator intents are realized through the available xApps and rApps. However, as the application catalog grows, realizing intents across a multi-vendor ecosystem raises three open problems. First, an intent must be mapped to the right xApps, rApps, APIs, and control procedures, but O-RAN defines application exposure rather than intent-driven selection and composition[3], leaving this step manual. Second, an intent may span multiple applications that depend on shared data, models, policies, and control actions, yet these dependencies are still validated in separate steps across design, deployment, and runtime. Third, an intent can trigger automated control, but conflict mitigation, authorization, lifecycle control, auditing, rollback, and human approval are not tied together into one governed execution path. These gaps make intent realization difficult to scale and govern in open, multi-vendor O-RAN systems.

To this end, this paper proposes AppWeaver, an agentic-AI framework that orchestrates end-to-end O-RAN workflows by using LLM-based agents to interpret operator intent, discover reusable xApps and rApps, reason about their capabilities and dependencies, and compose them into ordered, service-specific workflows. The main design elements and contributions of AppWeaver are summarized as follows.

- Agent Card for xApp/rApp. Each xApp and rApp is wrapped by a machine-readable Agent Card[4] that describes its functions, interfaces, control scope, resource demand, dependencies, policy constraints, and MCP tool endpoints. This allows agents to discover, compare, and reuse third-party applications without modifying their internals, while providing the semantic grounding needed for dependency reasoning and compatibility checking.
- Agentic workflow composition. Based on the Agent Card abstraction and the Agentic Resource Discovery (ARD) principle[5], LLM agents translate operator

Yongchao Dang and Riku Jäntti are with the Department of Information and Communications Engineering, School of Electrical Engineering, Aalto University, 02150 Espoo, Finland (e-mail: firstname.lastname@aalto.fi).

Tao Chen is with VTT Technical Research Centre of Finland, 02150 Espoo, Finland (e-mail: tao.chen@vtt.fi).

[1]https://www.o-ran.org/

[2]https://hexa-x-ii.eu/

[3]https://www.o-ran.org/specifications

[4]https://www.agentcard.net/

[5]https://agenticresourcediscovery.org/

TABLE I: Classification of representative agentic O-RAN systems by capability.

System	Control paradigm	Intent	Reuse & composition		Conflict & assurance			Evaluation
			Disc.	Comp.	Confl.	Valid.	Adapt.	
RIC-O [2]	Optimization-based RIC placement	✗	✗	✗	✗	✗	∼	Kubernetes
AutoRAN [3]	LLM-driven deployment automation	✓	✗	∼	✗	✓	∼	OAI/srsRAN/ARC
OREO [4]	Optimization-based xApp deployment	✗	∼	∼	∼	✗	✗	Colosseum emulator
RAN assurance [5]	Multi-agent assurance	∼	✗	∼	✓	✓	✓	Simulation
AGORAN [6]	Multi-agent intent negotiation	✓	∼	✗	✓	✗	∼	OAI/FlexRIC
AgentRAN [7]	Hierarchical multi-agent control	✓	✗	∼	∼	∼	✓	OAI (OTA)
Multi-scale [8]	Multi-timescale multi-agent control	✓	✗	✗	∼	✗	✓	Architecture
Multi-agentic rApp [9]	Multi-agent policy control	✓	✗	∼	✓	∼	∼	Simulation
MX-AI [10]	Multi-agent observability and control	✓	✗	∼	✗	∼	∼	OAI/FlexRIC
AppWeaver (ours)	Multi-agent workflow orchestration	✓	✓	✓	✓	✓	∼	OAI/FlexRIC

Control paradigm: control approach adopted. Intent: natural-language interface. Disc.: discovery of reusable xApps/rApps. Comp.: composition into executable workflows. Confl.: cross-application conflict handling. Valid.: pre-deployment sandbox or twin validation. Adapt.: runtime adaptation and rollback. Marks: ✓ demonstrated or core focus; ∼ partial or by-design; ✗ not addressed.

intent into workflows by discovering suitable xApps and rApps, reasoning about their capabilities and dependencies, and coordinating execution through A2A communication and a LangGraph-based state machine. This enables reusable O-RAN applications to be selected, connected, and orchestrated for different service requirements without manual workflow construction.

- Agentic workflow testing and deployment. Each generated workflow is tested before deployment through interface compatibility checking, policy verification, rollback planning, and human-in-the-loop approval when required. Only validated and approved workflows are deployed through standardized O-RAN interfaces, while operator policies, auditing, and rollback mechanisms keep execution observable, controlled, and reversible.

These agentic capabilities enable an intent-to-deployment pipeline that covers application discovery, workflow composition, predeployment validation, and governed execution across reusable O-RAN applications. The remainder of this paper is organized as follows. Section II reviews related work on agentic AI for O-RAN. Section III introduces the AppWeaver architecture. Section IV presents the prototype implementation and evaluation with OAI and FlexRIC. Section V concludes the paper and outlines future directions.

II. Agentic AI for O-RAN

Agentic AI for O-RAN refers to governed LLM-based agents that interpret operator intent, retrieve external knowledge, reason over network context, plan actions, and invoke O-RAN APIs and tools for automation. Unlike traditional rule-based network agents that follow predefined policies and fixed control logic, these agents interpret natural-language instructions and adapt their behavior to dynamic network conditions. Recent work explores this direction from three complementary perspectives, namely xApp and rApp development, O-RAN component deployment and lifecycle management, and intent-driven network automation, summarized in Table I.

A. Agentic AI for xApp and rApp Development

Developing xApps and rApps requires a heavy engineering burden, as each application requires RAN control logic, E2 and A1 service model integration, RIC SDK interfacing, and validation on the target platform. Agentic AI is being leveraged to reduce this burden, with LLM-based agents translating natural-language requirements into application code, service model bindings, configuration artifacts, and test templates, and supporting adaptive reasoning inside xApps for radio resource management [11]. However, these efforts mainly target the development of individual applications and treat the LLM as a code-generation or reasoning engine, while agentic capabilities such as tool invocation, knowledge retrieval, multi-step planning, and multi-agent collaboration remain underexplored.

B. Agentic AI for O-RAN Deployment and Management

O-RAN deployment and management involve placing disaggregated functions across cloud-native infrastructure, allocating resources, resolving inter-application conflicts, and verifying behavior before execution. To automate these tasks, existing studies combine optimization, learning-based prediction, and agentic decision-making. RIC-O optimizes the placement of distributed near-RT RIC instances across the cloud-edge continuum through dynamic clustering of radio nodes [2], while graph neural network based methods predict conflicts among xApps before deployment [12]. AutoRAN translates high-level operator intent into deployment, configuration, and testing artifacts for multi-vendor O-RAN systems [3], whereas

OREO formulates xApp deployment as a joint optimization problem over application selection, function sharing, configuration, and resource allocation [4]. Assurance-oriented frameworks add explicit verification stages to validate AI-generated decisions before execution [5]. Although these efforts reduce the operational complexity of deploying and managing O-RAN components, they mainly assume that the candidate applications are already known and focus on their placement, configuration, or validation, rather than on discovering reusable xApps and rApps and assembling them into service-specific workflows.

C. Agentic AI for Intent-Based RAN Automation

Intent-based RAN automation enables operators to specify high-level service objectives in natural language, which collaborating agents translate into control actions; a key challenge is how such intents are interpreted and resolved in multi-stakeholder settings. AGORAN addresses this challenge through intent negotiation, where LLM-based broker agents reconcile conflicting requirements under governance constraints and translate the agreed outcomes into enforceable policies through standard O-RAN interfaces [6]. Building on this direction, subsequent work aligns agent structures and computational models with O-RAN's hierarchical, multi-timescale control loops, as in AgentRAN, which coordinates rApp, xApp, and dApp agents through A2A communication and MCP-based tool invocation [7], and multi-scale frameworks that map LLMs, small language models (SLMs), and wireless foundation models to the Non-RT, Near-RT, and real-time loops to meet heterogeneous latency requirements [8]. The direction further extends to concrete implementations and system-level demonstrations, including conflict-aware policy generation through collaborative rApp agents [9], [13] and live prototypes such as MX-AI, which provides end-to-end observability and closed-loop control in operational deployments [10].

Despite advances in application development, deployment automation, intent understanding, architectural design, and closed-loop execution, existing systems commonly assume that the applications required to realize a service are already known. However, an end-to-end service often requires coordinating multiple reusable applications rather than executing a single xApp or rApp in isolation. For example, traffic steering typically involves KPI monitoring, anomaly detection, QoE prediction, traffic-steering optimization, and RAN control [14]. Current approaches therefore map operator intent to actions over predefined xApps and rApps, but do not address how such applications are discovered in an open, multi-vendor ecosystem, verified for interface and control-scope compatibility, and composed into validated executable workflows. As O-RAN application ecosystems expand and services increasingly span multiple vendors, this gap in application discovery and composition becomes a key barrier to fully autonomous service orchestration, motivating the design of AppWeaver.

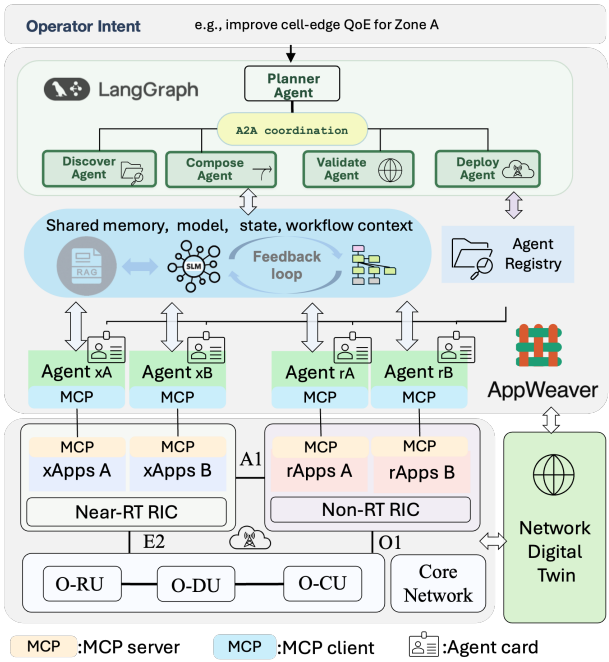


Fig. 1: AppWeaver architecture.

III. AppWeaver Architecture and Workflow

A. AppWeaver Architecture

Figure 1 shows the AppWeaver architecture, which sits between the operator specifying the intent and the RAN vendors providing the RIC and its xApps and rApps. AppWeaver does not replace existing O-RAN applications but orchestrates them into service-specific workflows through discovery, composition, validation, and governed execution. At its core, a LangGraph-based orchestration layer[6] runs a Planner agent that interprets the operator intent and coordinates four orchestration agents over the A2A bus[7]. The Discover agent identifies candidate xApps and rApps, the Compose agent assembles them into an ordered workflow, the Validate agent checks the workflow before deployment, and the Deploy agent applies the validated workflow to the RIC. These agents draw on two shared resources, an Agent Registry that stores the Agent Card and endpoint of each available xApp and rApp for capability-based selection, and a shared memory that holds the model state, running summaries, and workflow context across reasoning steps. Within this shared memory, retrieval-augmented generation (RAG) grounds discovery in the registry, a small language model (SLM) supports lightweight reasoning, and a feedback loop lets deployment outcomes refine later decisions.

[6]https://www.langchain.com/langgraph
[7]https://a2a-protocol.org/latest/specification/

AppWeaver operates across two environments, the live RAN and a network digital twin. On the live RAN, a per-application domain agent wraps each selected xApp or rApp and reaches it through an MCP client[8] connected to the MCP server co-located with the application in the Near-RT or Non-RT RIC. The server exposes monitoring and control functions as callable tools, so the domain agent orchestrates the workflow while the application executes the E2 and A1 control actions. In parallel, the Validate agent connects to a network digital twin that mirrors the per-layer RAN state as a live replica. Each candidate workflow is first exercised on the twin to estimate target KPI impact and detect conflicts or errors, so only validated workflows are applied to production. This separation gives AppWeaver a safety boundary by testing workflows before they reach the live RAN, while keeping all E2 and A1 control actions within existing xApps and rApps and their native control loops.

intent	Improve cell-edge QoE for Zone A; skill = analyze_qoe.
discover	RAG over Agent Registry → kpm-mon.xApp, qoe-pred.rApp, mobility-opt.rApp, qos-ctrl.xApp.
thought	Edge QoE unknown; read live KPIs first.
action	get_kpi { zone: "A" }
observe	{ prb_util: 0.87, sinr_edge_db: 3.1, edge_ues: 14 }
thought	Load high, edge SINR low; forecast QoE, find at-risk UEs.
action	predict_qoe { zone: "A" }
observe	{ qoe_mos: 2.8, at_risk_ues: 11, bottleneck: "edge_sinr" }
thought	Relieve edge: rebalance mobility, then raise priority.
action	optimize_handover { zone: "A" }; set_priority { class: "edge", weight: "high" }
observe	{ policy: "ho_margin +3 dB; steer 4 UEs → CS-7" }
validate	Twin what-if → { pred_qoe_mos: 3.6, conflicts: 0 }: pass.
deploy	Commit the validated policy over E2SM-RC to the live RAN; monitoring on.
result	Cell-edge QoE MOS 2.8 → 3.6; validated workflow deployed.

Fig. 2: AppWeaver ReAct trace for the cell-edge QoE intent.

B. AppWeaver Workflow

The AppWeaver workflow follows a reasoning-and-acting (ReAct) loop, illustrated in Fig. 2 for a cell-edge QoE intent, and moves through four stages, discovery, composition, validation, and deployment, all driven by the LLM-based agents in the AppWeaver core. Given an operator intent, the Planner agent interprets the request, extracts the target objective, constraints, and affected zone, and decomposes the intent into subgoals. The Discover agent then retrieves candidate applications from the Agent Registry, matching subgoals to advertised capabilities rather than names. For the cell-edge QoE objective (skill analyze_qoe), it selects a KPM-monitoring xApp for live radio KPIs, a QoE-prediction rApp for estimating user experience, a mobility-optimization rApp for adjusting handover behavior, and a QoS-control xApp for scheduling and priority changes.

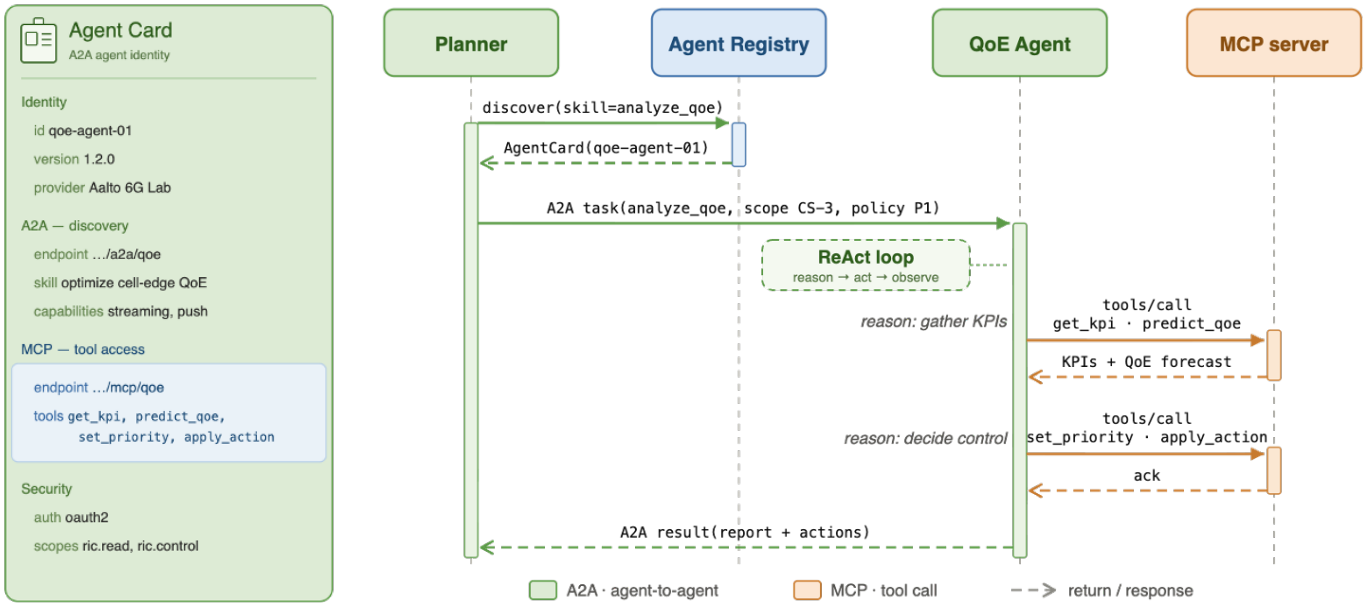
The Compose agent turns these applications, their tool schemas, and the operator constraints into an ordered workflow, which then executes under the same ReAct loop. As the trace in Fig. 2 shows, the loop first reads live KPIs for Zone A through the monitoring xApp, namely PRB utilization, edge SINR, and the edge UE count. It uses these observations to invoke the QoE-prediction rApp, estimating MOS and identifying at-risk UEs before any control decision. Guided by the prediction, it selects corrective actions, mobility rebalancing and higher scheduling priority for edge users, and maps them to the tools exposed by the mobility-optimization rApp and the QoS-control xApp. Each action, whether an A2A message between agents or an MCP tool call to an application, returns an observation that is written back into the shared state, so the workflow adapts step by step to the latest KPIs, predictions, and outcomes rather than running as a fixed pipeline. Finally, the Validate agent evaluates the candidate workflow on the digital twin and interprets its expected KPI impact and any conflicts, and once validation succeeds, the Deploy agent commits it to the live RAN with monitoring kept active.

[8]https://modelcontextprotocol.io/specification/

C. Agent Interfaces and Procedure

Figure 3 details the AppWeaver interfaces for exposing, discovering, and invoking third-party xApps and rApps, using the cell-edge QoE intent as an example. Fig. 3(a) shows the Agent Card, a machine-readable JSON descriptor registered in the Agent Registry that lets AppWeaver treat each xApp or rApp as a discoverable and reusable component. It specifies the application's capabilities, such as KPI monitoring, QoE prediction, mobility optimization, or QoS control; supported interfaces and data models, including E2SM-KPM, E2SM-RC, and A1 policy types; control scope over cells, slices, UEs, and tunable parameters; and resource requirements, operational constraints, and MCP tool endpoints used to monitor, configure, and control the RAN through the underlying application. Because this description is maintained outside the application, AppWeaver translates operator intent into RAN control through existing xApps and rApps without modifying their internal logic or driving O-RAN interfaces directly. A new application is onboarded by registering its Agent Card.

The MCP steps of Fig. 3(b) show how a domain agent invokes application tools through an MCP client connected to the MCP server co-located with its xApp or rApp. The agent first sends a tools/list request to discover available monitoring and control tools, then issues tools/call requests to invoke selected tools, such as get_kpi and predict_qoe for observation, and set_priority and apply_action for control. The MCP server translates these calls to the underlying application, and the xApp



(a) Agent Card

(b) Discovery and invocation sequence (A2A and MCP)

Fig. 3: AppWeaver agent interfaces and procedure.

or rApp executes RAN operation over E2 or A1. This keeps the agent at the orchestration level, hides vendor-specific management APIs behind a uniform tool interface, and makes each external operation, whether telemetry collection, invocation, or policy update, an explicit typed action that can be logged, validated, and replayed.

Using Agent Cards and MCP tools, the orchestration agents search the Agent Registry, select suitable domain agents, and assign tasks over the A2A coordination plane, as shown by the A2A steps of Fig. 3(b). For the cell-edge QoE intent, the Discover agent queries the registry for the required skill and receives matching Agent Cards, including the QoE agent and its endpoints. The Planner agent then assigns the QoE agent an A2A task carrying the objective, scope, and policy constraints. The QoE agent runs the ReAct loop in Fig. 2 by calling MCP tools to gather KPIs and a QoE forecast, reasoning over the results, applying selected control actions, and returning an A2A result with the report and actions to the Planner agent. This separation turns a high-level intent into concrete, traceable tool invocations without orchestration agents directly touching O-RAN interfaces.

The Agent Card and MCP abstractions expose heterogeneous xApps and rApps as discoverable, reusable agents, and the ReAct loop composes them through iterative reasoning, tool invocation, and observation into a validated workflow. AppWeaver serves as an intent-translation layer between the operator and vendor-provided RIC applications, mapping high-level objectives onto existing xApps and rApps through their agent interfaces. This layering separates timescales, with AppWeaver orchestrating at seconds to minutes while the applications retain native E2 and A1 control in their own loops, keeping orchestration separate from control execution.

IV. AppWeaver Evaluation

A. Prototype Setup

Fig. 4 shows the AppWeaver testbed, built on OpenAICellular[9]. The orchestration host runs the AppWeaver LangGraph orchestrator alongside the FlexRIC Near-RT RIC, and a separate GPU server hosts the gpt-oss planner[10] via Ollama[11]. The RAN includes a live network, with an OAI gNB, a 5G core, and RF-simulated UEs, and a RAN emulator that acts as a digital twin, with FlexRIC connected to both over E2 so the same xApps can run against the live network or the twin. Built on the micro-SDK abstraction of the RAN pipeline [15], the twin mirrors the live RAN through MAC, RLC, and PDCP state synchronization and exposes the same control-plane APIs, so composed workflows can be validated before deployment. Although OpenAICellular provides the base functionality for this testbed, it includes only a small set of xApps and rApps. To expand AppWeaver's application catalog, an agentic coding workflow based on OpenAI Codex[12] was used to develop four additional FlexRIC xApps in Python 3.10, covering KPM monitoring, QoE prediction, mobility optimization, and QoS control. rApps typically require a large, live-scale network, so in this testbed they are emulated as test endpoints with standardized APIs rather than deployed as production systems,

[9]https://www.openaicellular.org/
[10]https://huggingface.co/openai/gpt-oss-120b
[11]https://github.com/ollama/ollama
[12]https://openai.com/codex/

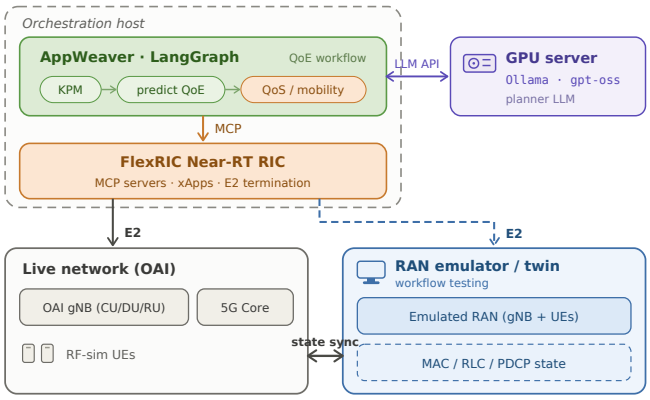


Fig. 4: AppWeaver prototype testbed.

and are left for future work in collaboration with industry partners.

B. Benchmark

1) Intent Benchmark: The intent benchmark consists of 50 operator requests, split into 11 known-world intents, for which a predefined workflow template exists, and 39 open-world intents, for which no template is available, with three single-capability cases among the known-world set. It evaluates AppWeaver's ability to translate high-level goals into correct xApp workflows. Each intent is a natural-language RIC request, such as "guarantee XR latency below 5 ms for the active slice", and is labeled with a ground-truth workflow defined as an ordered composition of four xApp capabilities. These capabilities are KPM for monitoring, SLICE for slice control, TC for transport control, and RC for RAN control validation. The intents reflect daily operator use cases across transport and QoS, anomaly diagnosis, slice management, latency assurance, enterprise services, capacity, mobility, reliability, and underspecified requests. The benchmark stresses three orchestration capabilities: template matching, which retrieves and executes the correct predefined workflow for known-world intents; open-world composition, which synthesizes a valid multi-xApp plan from Agent Cards when no template exists; and runtime robustness, which detects busy xApps, handles resource conflicts through safe queuing or blocking, and recovers without issuing unsafe calls.

2) Method Benchmark: AppWeaver is evaluated against three common intent-to-control baselines:

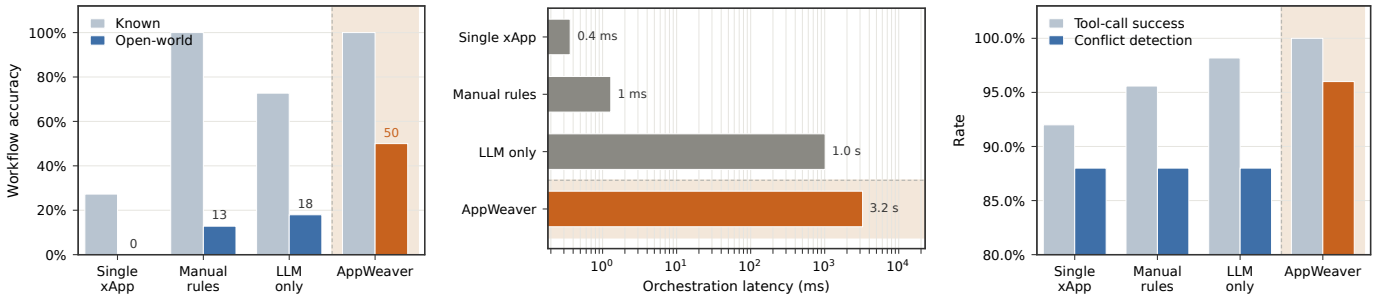
- Single xApp: selects and runs the single most relevant application for each intent, with no composition across applications. It is simple and fully deterministic, but by design cannot serve any intent whose objective spans more than one capability, and therefore establishes a lower bound for the benchmark.
- Manual rules: routes each intent to a fixed workflow through hand-crafted matching rules, for example mapping any latency-tagged intent to the KPM→SLICE→TC→RC template. This lifts the single-application ceiling and yields reliable multi-capability workflows for anticipated cases, but the rules must be written in advance and cannot generalize beyond their predefined set, forcing a fallback or failure whenever an intent matches no rule.
- LLM only: replaces fixed rules with an unconstrained language model that reads the operator intent and emits a sequence of applications and actions from its parametric knowledge. This removes the coverage limit, since the model can respond to previously unseen intents, but without discovery via Agent Cards its plans are ungrounded in the actual catalog, making them prone to referencing unavailable or incompatible applications.

C. Performance Evaluation

1) Evaluation Metrics: Each method is assessed along the following three axes.

- Workflow accuracy: measures whether a method composes the correct workflow. It is the fraction of intents whose composed workflow matches the ground-truth capability sequence, counted as correct only when all required capabilities appear in a valid order, and reported separately for known-world and open-world intents to distinguish template matching from open composition.
- Orchestration latency: measures how quickly a correct workflow is produced. It is the wall-clock time from intent submission to a deployable, validated workflow, covering planning, composition, and validation, but excluding the runtime of the RAN control actions themselves.
- Reliability: measures whether the produced workflow is safe to deploy. It combines the tool-call success rate, the incidence of hallucinated references to non-existent applications, and the detection rate for seeded cross-application conflicts, where methods without pre-deployment validation are credited for conflicts surfaced reactively at execution time, indicating whether every action is grounded in a real application and conflicts are caught before deployment.

2) Evaluation results: The evaluation in Fig. 5 compares AppWeaver with the benchmark baselines. As Fig. 5a shows, workflow accuracy separates the methods most clearly between known-world and open-world intents. On known-world intents, Manual rules and AppWeaver both reach 100%. Manual rules matches predefined templates, while AppWeaver recovers the same workflows through planner-driven orchestration. Single xApp reaches 27% by serving only single-capability cases, and LLM only reaches 73% by inferring some multi-step workflows but still selecting unavailable tools. These results follow from the benchmark design, since known-world intents are



(a) Workflow accuracy (b) Orchestration latency (c) Reliability

Fig. 5: AppWeaver versus baselines on workflow accuracy, orchestration latency, and reliability.

covered by predefined templates and most require multiple capabilities. Thus, Single xApp accuracy reflects only the share of single-capability intents. On open-world intents, all baselines degrade sharply. Single xApp cannot compose workflows and reaches 0%, Manual rules cannot cover unseen patterns and reaches 13%, and LLM only lacks grounding in the live catalog and reaches 18%. AppWeaver reaches 50% by grounding plans in Agent Card capabilities and validating them on the digital twin before deployment, which corresponds to an overall accuracy of roughly 61% across the full intent set, against 32% for the strongest baseline.

Despite its higher accuracy, AppWeaver has the highest orchestration latency, as shown in Fig. 5b, and the ordering across methods scales directly with the amount of model interaction in each working path. Single xApp and Manual rules respond fastest because they involve no inference at all and reduce to direct selection or template lookup. LLM only adds a single language-model planning pass. AppWeaver is the slowest because it interacts with the model repeatedly, as its ReAct loop performs Agent Card discovery, reasons over tool schemas, and composes the workflow step by step, after which the result is validated on the digital twin. This added latency is the cost of producing grounded and verified workflows, and it remains within the Non-RT timescale (>1 s)[13] for intent-driven orchestration.

The mechanisms that increase latency are also the main source of AppWeaver's reliability gains, as shown in Fig. 5c. AppWeaver achieves a 100% tool-call success rate because each workflow action is grounded in a discovered Agent Card, ensuring that plans reference only applications present in the catalog. In contrast, the baselines may invoke unavailable or mismatched tools when lookup rules or model knowledge diverge from the deployed catalog. The baselines detect a comparable share of conflicts only reactively, once execution fails, whereas AppWeaver exposes resource contention and interfering control actions proactively by validating each candidate workflow on the digital twin before deployment, enabling conflict detection and mitigation for safer orchestration of multi-xApp workflows on the live RAN and covering the runtime-robustness capability targeted by the benchmark.

[13]O-RAN.WG1.OAD, https://specifications.o-ran.org/.

V. Conclusion and Future Work

This article presented AppWeaver, an agentic-AI framework that translates operator intent into validated O-RAN workflows by discovering reusable xApps and rApps, composing them, and deploying them under operator governance. AppWeaver wraps each application in a machine-readable Agent Card for capability-based discovery, coordinates Planner, Discover, Compose, Validate, and Deploy agents over A2A within a LangGraph orchestrator, binds applications through MCP tool interfaces, and rehearses every candidate workflow on a network digital twin before it reaches the live RAN. By confining LLM agents to workflow-level orchestration, AppWeaver keeps all E2 and A1 control actions inside the native xApps, rApps, and RIC control loops, so reasoning and coordination sit above the RAN while execution remains within standardized interfaces under operator policy, auditing, and rollback. The prototype evaluation on an OAI and FlexRIC testbed supports this design. AppWeaver matches rule-based orchestration on templated intents, clearly outperforms all baselines on open-world intents where no template exists, and achieves fully grounded tool calls and the highest conflict detection, at an orchestration latency that remains within the Non-RT timescale for intent-driven automation.

Several directions remain for future work. First, runtime adaptation is currently partial, and tighter closed-loop self-healing together with learned digital-twin surrogates would strengthen the observe-and-adapt stage and reduce validation cost. Second, discovery is performed over a curated Agent Card catalog, and scaling to large multi-vendor repositories and open marketplaces raises trust, versioning, and conflict-resolution challenges. Third, rApps are emulated as test endpoints in the current testbed, and deploying AppWeaver against production-scale Non-RT RIC applications in collaboration with industry partners is a natural next step. Finally, the A2A and MCP bindings used here are pragmatic mechanisms, and aligning them with the RAN-grade requirements

on semantic contracts, authorization, behavioral bounds, traceability, and rollback now being studied in O-RAN working groups and 3GPP SA5 is important work toward a standardized agent-to-O-RAN interaction model.

References

[1] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia, "Beyond connectivity: An open architecture for AI-RAN convergence in 6G," IEEE Communications Magazine, vol. 64, no. 7, pp. 90–95, Jul. 2026.

[2] G. M. Almeida, G. Z. Bruno, A. Huff, M. Hiltunen, E. P. Duarte, C. B. Both, and K. V. Cardoso, "RIC-O: Efficient placement of a disaggregated and distributed RAN intelligent controller with dynamic clustering of radio nodes," IEEE Journal on Selected Areas in Communications, vol. 42, no. 2, pp. 446–459, Feb. 2024.

[3] S. Maxenti, R. Shirkhani, M. Elkael, L. Bonati, S. D'Oro, T. Melodia, and M. Polese, "AutoRAN: Automated and zero-touch open RAN systems," IEEE Transactions on Mobile Computing, vol. 25, no. 7, pp. 10 112–10 129, Jul. 2026.

[4] F. Mungari, C. Puligheddu, A. Garcia-Saavedra, and C. F. Chiasserini, "O-RAN intelligence orchestration framework for quality-driven xApp deployment and sharing," IEEE Transactions on Mobile Computing, vol. 24, no. 6, pp. 4811–4828, Jun. 2025.

[5] S. Singh, A. Bhat, Shweta M, S. K. Singh, M. Hong, Madhan Raj K, K. Sithamparanathan, S. A. Khowaja, and K. Dev, "Agentic AI for ultra-modern networks: Multi-agent framework for RAN autonomy and assurance," in Proc. IEEE Int. Conf. Commun. (ICC), Glasgow, United Kingdom, May 2026, pp. 1–6.

[6] I. Chatzistefanidis, N. Nikaein, A. Leone, A. Maatouk, L. Tassiulas, R. Morabito, I. Pitsiorlas, and M. Kountouris, "AGORAN: An agentic open marketplace for 6G RAN automation," Computer Networks, vol. 275, p. 111927, Feb. 2026.

[7] M. Elkael, S. D'Oro, L. Bonati, M. Polese, Y. Lee, K. Furueda, and T. Melodia, "AgentRAN: An agentic AI architecture for autonomous control of open 6G networks," IEEE Communications Magazine, 2026, early access.

[8] H. Navidan, M. Cheraghinia, J. Fontaine, M. Seif, E. De Poorter, H. V. Poor, I. Moerman, and A. Shahid, "Toward autonomous O-RAN: A multi-scale agentic AI framework for real-time network control and management," IEEE Network, 2026, early access.

[9] H. Li, Y. Wu, and D. Simeonidou, "Multi-agentic AI for conflict-aware rApp policy orchestration in open RAN," in Proc. IEEE Int. Conf. Commun. (ICC). Glasgow, United Kingdom: IEEE, May 2026, pp. 1–6.

[10] I. Chatzistefanidis, A. Leone, A. Yaghoubian, M. Irazabal, N. Sehad, L. Bariah, M. Debbah, and N. Nikaein, "MX-AI: Agentic observability and control platform for open and AI-RAN," in Proc. IEEE Int. Conf. Commun. (ICC), Glasgow, United Kingdom, May 2026, pp. 1–6.

[11] A. Maatouk, N. Piovesan, F. Ayed, A. De Domenico, and M. Debbah, "Large language models for telecom: Forthcoming impact on the industry," IEEE Communications Magazine, vol. 63, no. 1, pp. 62–68, Jan. 2025.

[12] M. Al Shami, J. Yan, and E. T. Fapi, "O-RAN xApps conflict prediction using graph convolutional networks," IEEE Transactions on Mobile Computing, vol. 25, no. 7, pp. 10 876–10 892, Jul. 2026.

[13] S. Chatzimiltis, M. B. Mashhadi, M. Shojafar, M. Debbah, and R. Tafazolli, "Agentic AI for 6G: A new paradigm for autonomous RAN security compliance," IEEE Communications Standards Magazine, 2026, early access.

[14] H. Yang, C. Mcpeak, R. Nagampally, N. Tripathi, G. Anderson, J. H. Reed, Y. Yang, and D. J. Jakubisin, "Agile 5G networks: Advance traffic steering xApp for interference mitigation," in Proc. IEEE Military Commun. Conf. (MILCOM). Washington, DC, USA: IEEE, Oct. 2024, pp. 300–305.

[15] R. Schmidt and N. Nikaein, "RAN engine: Service-oriented RAN through containerized micro-services," IEEE Transactions on Network and Service Management, vol. 18, no. 1, pp. 469–481, Mar. 2021.